

Due Date: 09/27/2026 @ 11:55 PM

N-Gram Language Models: Assignment 2

In the class, we defined the language modeling as the task of predicting the next word in a sequence given the previous words. In this assignment, we will focus on the related problem of predicting the next *character* in a sequence given the previous characters.

The learning goals of this assignment are to:

- Understand how to compute language model probabilities using maximum likelihood estimation.
- Implement basic smoothing, back-off and interpolation.
- Have fun using a language model to probabilistically generate texts.
- Use a set of language models to perform text classification.

Here are the materials that you should download for this assignment:

- Skeleton python code.
- training data for text classification task.
- dev data for text classification task.
- test file for leaderboard

This assignment has several deliverables:

- Your implementations for the functions in the skeleton code.
- Your model's output for the test set (your model will be ranked on a leaderboard against the other students' outputs)
- A demonstration YouTube video showcasing functional code and the resulting output
- Your writeup about the homework. [Here is an example writeup that illustrates what I am looking for.](#)

Part 0: Generating N-Grams

Write a function `ngrams(n, text)` that produces a list of all n-grams of the specified size from the input text. Each n-gram should consist of a 2-element tuple `(context, char)`, where the context is itself an n-length string comprised of the n characters preceding the current character. The sentence should be padded with n ~ characters at the beginning (I've provided you with `start_pad(n)` for this purpose). If $n=0$, all contexts should be empty strings. You may assume that $n \geq 0$.

```
>>> ngrams(1, 'abc')
[('~', 'a'), ('a', 'b'), ('b', 'c')]

>>> ngrams(2, 'abc')
[('~~', 'a'), ('~a', 'b'), ('ab', 'c')]
```

I've also given you the function `create_ngram_model(model_class, path, n, k)` that will create and return an n-gram model trained on the entire file path provided

Due Date: 09/27/2026 @ 11:55 PM

N-Gram Language Models: Assignment 2

and `create_ngram_model_lines(model_class, path, n, k)` that will create and return an n-gram model trained line-by-line on the file path provided. You should use the first for the Shakespeare file and the second for the city name files.

Part 1: Creating an N-Gram Model

In this section, you will build a simple n-gram language model that can be used to generate random text resembling a source document. Your use of external code should be limited to built-in Python modules, which excludes, for example, NumPy and NLTK.

1. In the `NgramModel` class, write an initialization method `__init__(self, n, k)` which stores the order `n` of the model and initializes any necessary internal variables. Then write a method `get_vocab(self)` that returns the vocab (this is the set of all characters used by this model).
2. Write a method `update(self, text)` which computes the n-grams for the input sentence and updates the internal counts. Also write a method `prob(self, context, char)` which accepts an n-length string representing a context and a character, and returns the probability of that character occurring, given the preceding context. If you encounter a novel `context`, the probability of any given `char` should be $1/V$ where V is the size of the vocab.

```
>>> m = NgramModel(1, 0)
>>> m.update('abab')
>>> m.get_vocab()
{'b', 'a'}
>>> m.update('abcd')
>>> m.get_vocab()
{'b', 'a', 'c', 'd'}
>>> m.prob('a', 'b')
1.0
>>> m.prob('~', 'c')
0.0
>>> m.prob('b', 'c')
0.5
```

3. Write a method `random_char(self, context)` which returns a random character according to the probability distribution determined by the given context. Specifically, let $V = \langle v_1, v_2, \dots, v_n \rangle$ be the vocab, sorted according to Python's natural lexicographic ordering, and let $0 \leq r < 1$ be a random number between 0 and 1. Your method should return the character v_i such that

Due Date: 09/27/2026 @ 11:55 PM

N-Gram Language Models: Assignment 2

$$\sum_{j=1}^{i-1} P(v_j \mid \text{context}) \leq r < \sum_{j=1}^i P(v_j \mid \text{context}).$$

You should use a single call to the `random.random()` function to generate r .

```
>>> m = NgramModel(0, 0)
>>> m.update('abab')
>>> m.update('abcd')
>>> random.seed(1)
>>> [m.random_char('') for i in range(25)]
['a', 'c', 'c', 'a', 'b', 'b', 'b', 'c', 'a', 'a', 'c', 'b', 'c', 'a', 'b', 'b', 'a', 'd', 'd', 'a', 'a', 'b', 'd', 'b', 'a']
```

4. In the `NgramModel` class, write a method `random_text(self, length)` which returns a string of characters chosen at random using the `random_char(self, context)` method. Your starting context should always be n characters, and the context should be updated as characters are generated. If $n=0$, your context should always be the empty string. You should continue generating characters until you've produced the specified number of random characters, then return the full string.

```
>>> m = NgramModel(1, 0)
>>> m.update('abab')
>>> m.update('abcd')
>>> random.seed(1)
>>> m.random_text(25)
abcdbabcbcdabababcbddabcbda
```

Writing Shakespeare

Now you can train a language model. First grab some text like this corpus of Shakespeare:

```
$ wget http://cs.stanford.edu/people/karpathy/char-rnn/shakespeare_input.txt
```

Try generating some Shakespeare with different order n-gram models. You should try running the following commands:

```
>>> m = create_ngram_model(NgramModel, 'shakespeare_input.txt', 2)
>>> m.random_text(250)
```

Due Date: 09/27/2026 @ 11:55 PM

N-Gram Language Models: Assignment 2

```
>>> m = create_ngram_model(NgramModel, 'shakespeare_input.txt', 3)
>>> m.random_text(250)

>>> m = create_ngram_model(NgramModel, 'shakespeare_input.txt', 4)
>>> m.random_text(250)

>>> m = create_ngram_model(NgramModel, 'shakespeare_input.txt', 7)
>>> m.random_text(250)
```

What do you think? Is it as good as [1000 monkeys working at 1000 typewriters](#)?

After generating a bunch of short passages, do you notice anything? *They all start with F!* In fact, after we hit a certain order, the first word is always *First*? Why is that? Is the model trying to be clever? Is it saying *First, I will generate the word "First"*? No, probably not. Explain what is going on in your writeup.

Part 2: Perplexity, Smoothing, and Interpolation

In this part of the assignment, you'll adapt your code in order to implement several of the techniques described in [Section 4.2 of the Jurafsky and Martin textbook](#).

Perplexity

How do we know whether a language model is good? There are two basic approaches:

1. Task-based evaluation (also known as extrinsic evaluation), where we use the language model as part of some other task, like automatic speech recognition, or spelling correction, or an OCR system that tries to convert a professor's messy handwriting into text.
2. Intrinsic evaluation. Intrinsic evaluation tries to directly evaluate the goodness of the language model by seeing how well the probability distributions that it estimates are able to explain some previously unseen test set.

Here's what the textbook says:

For an intrinsic evaluation of a language model we need a test set. As with many of the statistical models in our field, the probabilities of an N-gram model come from the corpus it is trained on, the training set or training corpus. We can then measure the quality of an N-gram model by its performance on some unseen data called the test set or test corpus. We will also sometimes call test sets and other datasets that are not in our training sets held out corpora because we hold them out from the training data.

Due Date: 09/27/2026 @ 11:55 PM

N-Gram Language Models: Assignment 2

So if we are given a corpus of text and want to compare two different N-gram models, we divide the data into training and test sets, train the parameters of both models on the training set, and then compare how well the two trained models fit the test set.

But what does it mean to “fit the test set”? The answer is simple: whichever model assigns a higher probability to the test set is a better model.

We'll implement the most common method for intrinsic metric of language models: *perplexity*. The perplexity of a language model on a test set is the inverse probability of the test set, normalized by the number of characters. For a test set $W=w_1w_2\dots w_N$:

$$\begin{aligned} \text{Perplexity}(W) &= P(w_1w_2\dots w_N)^{-\frac{1}{N}} \\ &= \sqrt[N]{\frac{1}{P(w_1w_2\dots w_N)}} \\ &= \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_1\dots w_{i-1})}} \end{aligned}$$

Now implement the `perplexity(self, text)` function in `NgramModel`. A couple of things to keep in mind:

1. Numeric underflow is going to be a problem, so consider using logs.
2. Perplexity is undefined if the language model assigns any zero probabilities to the test set. In that case your code should return positive infinity - `float('inf')`.
3. On your unsmoothed models, you'll definitely get some zero probabilities for the test set. To test you code, you should try computing perplexity on the training set, and you should compute perplexity for your language models that use smoothing and interpolation.

In your report, discuss the perplexity for text that is similar and different from Shakespeare's plays. I provided you [two dev text files](#), a New York Times article and several of Shakespeare's sonnets, but feel free to experiment with your own text.

```
>>> m = NgramModel(1, 0)
>>> m.update('abab')
>>> m.update('abcd')
>>> m.perplexity('abcd')
1.189207115002721
>>> m.perplexity('abca')
inf
>>> m.perplexity('abcda')
```

Due Date: 09/27/2026 @ 11:55 PM

N-Gram Language Models: Assignment 2

1.515716566510398

Note: you may want to create a smoothed language model before calculating perplexity on real data.

Smoothing

Laplace Smoothing is described in section 4.4.1. Laplace smoothing adds one to each count (hence its alternate name *add-one smoothing*). Since there are V characters in the vocabulary and each one was incremented, we also need to adjust the denominator to take into account the extra V observations.

$$P_{Laplace}(w_i) = \frac{count_i + 1}{N + |V|}$$

A variant of Laplace smoothing is called *Add-k smoothing* or *Add-epsilon smoothing*. This is described in section Add-k 4.4.2. Update your `NgramModel` code from Part 1 to implement add-k smoothing.

```
>>> m = NgramModel(1, 1)
>>> m.update('abab')
>>> m.update('abcd')
>>> m.prob('a', 'a')
0.14285714285714285
>>> m.prob('a', 'b')
0.5714285714285714
>>> m.prob('c', 'd')
0.4
>>> m.prob('d', 'a')
0.25
```

Interpolation

The idea of interpolation is to calculate the higher order n-gram probabilities also combining the probabilities for lower-order n-gram models. Like smoothing, this helps us avoid the problem of zeros if we haven't observed the longer sequence in our training data. Here's the math:

$$P_{interpolation}(w_i|w_{i-2}w_{i-1}) = \lambda_1 P(w_i|w_{i-2}w_{i-1}) + \lambda_2 P(w_i|w_{i-1}) + \lambda_3 P(w_i)$$

where $\lambda_1 + \lambda_2 + \lambda_3 = 1$

Due Date: 09/27/2026 @ 11:55 PM

N-Gram Language Models: Assignment 2

I've provided you with another class definition `NgramModelWithInterpolation` that extends `NgramModel` for you to implement interpolation. If you've written your code robustly, you should only need to override the `get_vocab(self)`, `update(self, text)`, and `prob(self, context, char)` methods, along with the initializer.

The value of n passed into `__init__(self, n, k)` is the highest order n-gram to be considered by the model (e.g. $n=2$ will consider 3 different length n-grams). Add-k smoothing should take place only when calculating the individual order n-gram probabilities, not when calculating the overall interpolation probability.

By default set the lambdas to be equal weights, but you should also write a helper function that can be called to overwrite this default. Setting the lambdas in the helper function can either be done heuristically or by using a development set, but in the example code below, we've used the default.

```
>>> m = NgramModelWithInterpolation(1, 0)
>>> m.update('abab')
>>> m.prob('a', 'a')
0.25
>>> m.prob('a', 'b')
0.75

>>> m = NgramModelWithInterpolation(2, 1)
>>> m.update('abab')
>>> m.update('abcd')
>>> m.prob('~a', 'b')
0.4682539682539682
>>> m.prob('ba', 'b')
0.4349206349206349
>>> m.prob('~c', 'd')
0.27222222222222225
>>> m.prob('bc', 'd')
0.3222222222222222
```

In your report, experiment with a few different lambdas and values of k and discuss their effects.

Part 3: Text Classification using N-Grams

Due Date: 09/27/2026 @ 11:55 PM

N-Gram Language Models: Assignment 2

Language models can be applied to text classification. If we want to classify a text D into a category $c \in C = C_1, \dots, C_N$. We can pick the category c that has the largest posterior probability given the text. That is,

$$c^* = \operatorname{argmax}_{c \in C} P(c|D)$$

Using Bayes rule, this can be rewritten as:

$$c^* = \operatorname{argmax}_{c \in C} P(D|c)P(c)$$

If we assume that all classes are equally likely, then we can just drop the $P(c)$ term:

$$= \operatorname{argmax}_{c \in C} P(D|c)$$

Here $P(D|c)$ is the likelihood of D under category c , which can be computed by training language models for all texts associated with category c . This technique of text classification is drawn from [literature on authorship identification](#), where the approach is to learn a separate language model for each author, by training on a data set from that author. Then, to categorize a new text D , they use each language model to calculate the likelihood of D under that model, and pick the category that assigns the highest probability to D .

Try it! I have provided you training and validation datasets consisting of the names of cities. The task is to predict the country a city is in. The following countries are included in the dataset.

af	Afghanistan
cn	China
de	Germany
fi	Finland
fr	France
in	India
ir	Iran
pk	Pakistan
za	South Africa

We'll set up a leaderboard for the text classification task. Your job is to configure a set of language models that perform the best on the text classification task. We will use the city names dataset,

Due Date: 09/27/2026 @ 11:55 PM

N-Gram Language Models: Assignment 2

which you should have already downloaded. The test set has one unlabeled city name per line. Your code should output a file `test_labels.txt` with one two-letter country code per line.

Feel free to extend the `NgramModel` or `NgramModelWithInterpolation` when creating your language model. Possible ideas to consider and experiment with when creating your model are utilizing a special end-of-text character, trying a new method for determining the vocab, and improving how your model handles novel characters.

In your report, describe the parameters of your final leaderboard model and any experimentation you did before settling on it.

Deliverables

Here are the deliverables that you will need to submit:

- `writeup.pdf`
- A demonstration YouTube video showcasing functional code and the resulting output
- `ngram_skeleton.py`
- `test_labels.txt` predictions for leaderboard.